

# Business Case Study

## Delhivery- Feature Engineering

Submitted By: Heena Khan

### Colab Link:

<https://drive.google.com/file/d/1Ktm5CcD0yPAdb8vRkKo3W-VnovRYdRG1/view?usp=sharing>

### About Delhivery

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities.

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

### Problem Statement

Delhivery wants to analyze logistics and trip-level delivery data generated from engineering pipelines.

The main goals are:

- Clean and process raw logistics data
- Merge fragmented shipment records into meaningful trip-level information
- Perform exploratory data analysis (EDA)
- Engineer useful features
- Compare actual delivery performance vs OSRM estimated performance
- Detect and treat missing values and outliers
- Prepare the dataset for forecasting and machine learning models
- Generate actionable business insights and recommendations

### Import Required Libraries

```
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Statistics
from scipy.stats import ttest_ind, ttest_rel

# Preprocessing
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

### Load Dataset

```
df = pd.read_csv("/content/delhivery_data.csv")
```

## Basic Exploration

### Shape of Data

```
df.shape
```

```
(21039, 24)
```

### First 5 Rows

```
df.head()
```

|   | data     | trip_creation_time         | route_schedule_uuid                              | route_type | trip_uuid               | source_center | source_name                | destination_center | destination_name              | od_start_time              | ... | cutoff_timestamp | actual_distance_to_destination | actual_time | osrm_time | osrm_distance | factor   | segment_actual_time | segment_osrm_time | segment_osrm_distance | segment_factor |
|---|----------|----------------------------|--------------------------------------------------|------------|-------------------------|---------------|----------------------------|--------------------|-------------------------------|----------------------------|-----|------------------|--------------------------------|-------------|-----------|---------------|----------|---------------------|-------------------|-----------------------|----------------|
| 0 | training | 2018-09-20 02:35:36.476840 | thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3.. | Carting    | trip-153741093647649320 | IND388121AAA  | Anand_VUNagar_DC (Gujarat) | IND388620AAB       | Khambhat_MotvdDPP_D (Gujarat) | 2018-09-20 03:21:32.418600 | ... | 20-09-2018 4.27  | 10.435660                      | 14.0        | 11.0      | 11.9653       | 1.272727 | 14.0                | 11.0              | 11.9653               | 1.272727       |
| 1 | training | 2018-09-20 02:35:36.       | thanos::sroute:eb7bfc78-b351-4c0e-a951-          | Carting    | trip-153741093647649320 | IND388121AAA  | Anand_VUNagar_DC (Gujarat) | IND388620AAB       | Khambhat_MotvdDPP_D (Gujarat) | 2018-09-20 03:21:32.418600 | ... | 20-09-2018 4.17  | 18.936842                      | 24.0        | 20.0      | 21.7243       | 1.200000 | 10.0                | 9.0               | 9.7590                | 1.111111       |

|   | data     | trip_creation_time         | route_schedule_uuid                              | route_type | trip_uuid               | source_center | source_name                | destination_center | destination_name               | od_start_time              | ... | cutoff_timestamp           | actual_distance_to_destination | actual_time | osrm_time | osrm_distance | factor   | segment_actual_time | segment_osrm_time | segment_osrm_distance | segment_factor |
|---|----------|----------------------------|--------------------------------------------------|------------|-------------------------|---------------|----------------------------|--------------------|--------------------------------|----------------------------|-----|----------------------------|--------------------------------|-------------|-----------|---------------|----------|---------------------|-------------------|-----------------------|----------------|
|   |          | 476840                     | fa3d5c3..                                        |            |                         |               |                            |                    |                                |                            |     |                            |                                |             |           |               |          |                     |                   |                       |                |
| 2 | training | 2018-09-20 02:35:36.476840 | thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3.. | Carting    | trip-153741093647649320 | IND388121AAA  | Anand_VUNagar_DC (Gujarat) | IND388620AAB       | Khambhat_Motvd DPP_D (Gujarat) | 2018-09-20 03:21:32.418600 | ... | 2018-09-20 04:01:19.505586 | 27.637279                      | 40.0        | 28.0      | 32.5395       | 1.428571 | 16.0                | 7.0               | 10.8152               | 2.285714       |
| 3 | training | 2018-09-20 02:35:36.476840 | thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3.. | Carting    | trip-153741093647649320 | IND388121AAA  | Anand_VUNagar_DC (Gujarat) | IND388620AAB       | Khambhat_Motvd DPP_D (Gujarat) | 2018-09-20 03:21:32.418600 | ... | 20-09-2018 3.39            | 36.118028                      | 62.0        | 40.0      | 45.5620       | 1.550000 | 21.0                | 12.0              | 13.0224               | 1.750000       |
| 4 | training | 2018-09-20 02:35:36.476840 | thanos::sroute:eb7bfc78-b351-4c0e-a951-          | Carting    | trip-153741093647649320 | IND388121AAA  | Anand_VUNagar_DC (Gujarat) | IND388620AAB       | Khambhat_Motvd DPP_D (Gujarat) | 2018-09-20 03:21:32.418600 | ... | 20-09-2018 3.33            | 39.386040                      | 68.0        | 44.0      | 54.2181       | 1.545455 | 6.0                 | 5.0               | 3.9153                | 1.200000       |

|  | data | trip_creation_time | route_schedule_uuid | route_type | trip_uuid | source_center | source_name | destination_center | destination_name | od_start_time | ... | cutoff_timestamp | actual_distance_to_destination | actual_time | osrm_time | osrm_distance | factor | segment_actual_time | segment_osrm_time | segment_osrm_distance | segment_factor |
|--|------|--------------------|---------------------|------------|-----------|---------------|-------------|--------------------|------------------|---------------|-----|------------------|--------------------------------|-------------|-----------|---------------|--------|---------------------|-------------------|-----------------------|----------------|
|  |      |                    | fa3d5c3..           |            |           |               |             |                    |                  |               |     |                  |                                |             |           |               |        |                     |                   |                       |                |

5 rows × 24 columns

# Data Types

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 63060 entries, 0 to 63059
Data columns (total 24 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   data                                     63060 non-null  object
1   trip_creation_time                     63060 non-null  object
2   route_schedule_uuid                   63060 non-null  object
3   route_type                             63060 non-null  object
4   trip_uuid                              63060 non-null  object
5   source_center                          63060 non-null  object
6   source_name                            62927 non-null  object
7   destination_center                    63060 non-null  object
8   destination_name                       62964 non-null  object
9   od_start_time                         63060 non-null  object
10  od_end_time                           63060 non-null  object
11  start_scan_to_end_scan                 63060 non-null  int64
12  is_cutoff                              63060 non-null  bool
13  cutoff_factor                          63060 non-null  int64
14  cutoff_timestamp                       63060 non-null  object
15  actual_distance_to_destination         63060 non-null  float64
16  actual_time                            63060 non-null  int64
17  osrm_time                              63060 non-null  int64
18  osrm_distance                          63060 non-null  float64
19  factor                                 63060 non-null  float64
20  segment_actual_time                    63059 non-null  float64
21  segment_osrm_time                      63059 non-null  float64
22  segment_osrm_distance                  63059 non-null  float64
23  segment_factor                         63059 non-null  float64
dtypes: bool(1), float64(7), int64(4), object(12)
memory usage: 11.1+ MB
```

# Statistical Summary

| df.describe()         |                                |                       |                                        |                      |                      |                        |                      |                             |                               |                               |                        |
|-----------------------|--------------------------------|-----------------------|----------------------------------------|----------------------|----------------------|------------------------|----------------------|-----------------------------|-------------------------------|-------------------------------|------------------------|
|                       | start_sc<br>an_to_e<br>nd_scan | cuto<br>ff_fa<br>ctor | actual_dist<br>ance_to_de<br>stination | actu<br>al_ti<br>me  | osr<br>m_ti<br>me    | osrm<br>_dist<br>_ance | facto<br>r           | segmen<br>t_actua<br>l_time | segme<br>nt_osr<br>m_tim<br>e | segment<br>_osrm_d<br>istance | segm<br>ent_f<br>actor |
| c<br>o<br>u<br>n<br>t | 63060.00<br>0000               | 6306<br>0.00<br>0000  | 63060.0000<br>00                       | 6306<br>0.00<br>0000 | 6306<br>0.00<br>0000 | 6306<br>0.000<br>000   | 6306<br>0.00<br>0000 | 63059.0<br>00000            | 63059.<br>000000              | 63059.00<br>000               | 6305<br>9.000<br>000   |
| m<br>e<br>a<br>n      | 945.6445<br>61                 | 229.<br>1842<br>69    | 230.335283                             | 410.<br>0734<br>70   | 210.<br>7734<br>38   | 280.6<br>0854<br>2     | 2.10<br>4299         | 36.0836<br>36               | 18.507<br>747                 | 22.84098                      | 2.216<br>878           |
| st<br>d               | 1018.104<br>546                | 337.<br>5926<br>35    | 337.806405                             | 585.<br>3420<br>66   | 302.<br>2467<br>07   | 413.2<br>2316<br>8     | 1.47<br>3226         | 51.3443<br>48               | 14.340<br>125                 | 17.26405                      | 5.450<br>390           |
| m<br>in               | 20.00000<br>0                  | 9.00<br>0000          | 9.000267                               | 9.00<br>0000         | 6.00<br>0000         | 9.072<br>900           | 0.20<br>3125         | -<br>211.000<br>000         | 0.0000<br>00                  | 0.00000                       | -<br>23.44<br>4444     |

|     | start_scan_to_end_scan | cutoff_factor | actual_distance_to_destination | actual_time | osrm_time   | osrm_distance | factor    | segment_actual_time | segment_osrm_time | segment_osrm_distance | segment_factor |
|-----|------------------------|---------------|--------------------------------|-------------|-------------|---------------|-----------|---------------------|-------------------|-----------------------|----------------|
| 25% | 164.000000             | 22.000000     | 23.440175                      | 52.000000   | 27.000000   | 30.123350     | 1.609004  | 20.000000           | 11.000000         | 12.132700             | 1.350000       |
| 50% | 455.000000             | 66.000000     | 66.247675                      | 134.000000  | 65.000000   | 79.419200     | 1.857143  | 29.000000           | 17.000000         | 23.541700             | 1.684211       |
| 75% | 1546.000000            | 286.000000    | 286.514187                     | 507.000000  | 254.000000  | 339.747250    | 2.215385  | 40.000000           | 22.000000         | 27.843650             | 2.250000       |
| max | 4535.000000            | 1927.000000   | 1927.447705                    | 4532.000000 | 1686.000000 | 2326.199100   | 77.387097 | 2464.000000         | 997.000000        | 1367.592800           | 574.250000     |

## Missing Values

```
df.isnull().sum()
```

|                                |     |
|--------------------------------|-----|
|                                | 0   |
| data                           | 0   |
| trip_creation_time             | 0   |
| route_schedule_uuid            | 0   |
| route_type                     | 0   |
| trip_uuid                      | 0   |
| source_center                  | 0   |
| source_name                    | 133 |
| destination_center             | 0   |
| destination_name               | 96  |
| od_start_time                  | 0   |
| od_end_time                    | 0   |
| start_scan_to_end_scan         | 0   |
| is_cutoff                      | 0   |
| cutoff_factor                  | 0   |
| cutoff_timestamp               | 0   |
| actual_distance_to_destination | 0   |
| actual_time                    | 0   |
| osrm_time                      | 0   |
| osrm_distance                  | 0   |
| factor                         | 0   |
| segment_actual_time            | 1   |
| segment_osrm_time              | 1   |
| segment_osrm_distance          | 1   |

|                |   |
|----------------|---|
|                | 0 |
| segment_factor | 1 |

dtype: int64

## Convert Datetime Columns

```
datetime_cols = [
    'trip_creation_time',
    'od_start_time',
    'od_end_time',
    'cutoff_timestamp'
]

# Define the known formats for 'cutoff_timestamp'
# The order matters: try the more specific or less common format first
# if there's overlap
cutoff_timestamp_formats = [
    '%d-%m-%Y %H.%M',          # For '20-09-2018 4.27'
    '%Y-%m-%d %H:%M:%S.%f'    # For '2018-09-20 04:01:19.505586'
]

for col in datetime_cols:
    if col == 'cutoff_timestamp':
        df[col] = pd.to_datetime(df[col], format=cutoff_timestamp_formats,
                                errors='coerce')
    else:
        # For other datetime columns, let pandas infer if they are
        # consistent, or handle with coerce
        df[col] = pd.to_datetime(df[col], errors='coerce')
```

## Handle Missing Values

### Percentage Missing

```
missing_percent = (df.isnull().sum()/len(df))*100
missing_percent
```

|                     |          |
|---------------------|----------|
|                     | 0        |
| data                | 0.000000 |
| trip_creation_time  | 0.000000 |
| route_schedule_uuid | 0.000000 |
| route_type          | 0.000000 |
| trip_uuid           | 0.000000 |
| source_center       | 0.000000 |
| source_name         | 0.210910 |
| destination_center  | 0.000000 |
| destination_name    | 0.152236 |
| od_start_time       | 0.000000 |
| od_end_time         | 0.000000 |

|                                |           |
|--------------------------------|-----------|
|                                | <b>0</b>  |
| start_scan_to_end_scan         | 0.000000  |
| is_cutoff                      | 0.000000  |
| cutoff_factor                  | 0.000000  |
| cutoff_timestamp               | 97.656200 |
| actual_distance_to_destination | 0.000000  |
| actual_time                    | 0.000000  |
| osrm_time                      | 0.000000  |
| osrm_distance                  | 0.000000  |
| factor                         | 0.000000  |
| segment_actual_time            | 0.001586  |
| segment_osrm_time              | 0.001586  |
| segment_osrm_distance          | 0.001586  |
| segment_factor                 | 0.001586  |

dtype: float64

## Fill or Drop

```
# Drop rows where source or destination is missing
df.dropna(subset=['source_name', 'destination_name'], inplace=True)

# Fill missing values in route_type
df['route_type'] = df['route_type'].fillna(
    df['route_type'].mode()[0]
)
```

## Understand the Granularity

**One shipment/trip is divided into multiple rows.**

We need to:

- Merge rows at:
  - Trip + Source + Destination level
- Then aggregate at:
  - Trip level

## Aggregation

### Aggregate Using:



## Feature Engineering

### Extract Features from Source/Destination

```
trip['source_name'].head()
```

|   | source_name                             |
|---|-----------------------------------------|
| 0 | Surat_Central_D_12 (Gujarat)            |
| 1 | Delhi_Lajpat_IP (Delhi)                 |
| 2 | FBD_Balabhgarh_DPC (Haryana)            |
| 3 | Gulbarga_Nehrugnj_I (Karnataka)         |
| 4 | Narsinghpur_KndliDPP_D (Madhya Pradesh) |

dtype: object

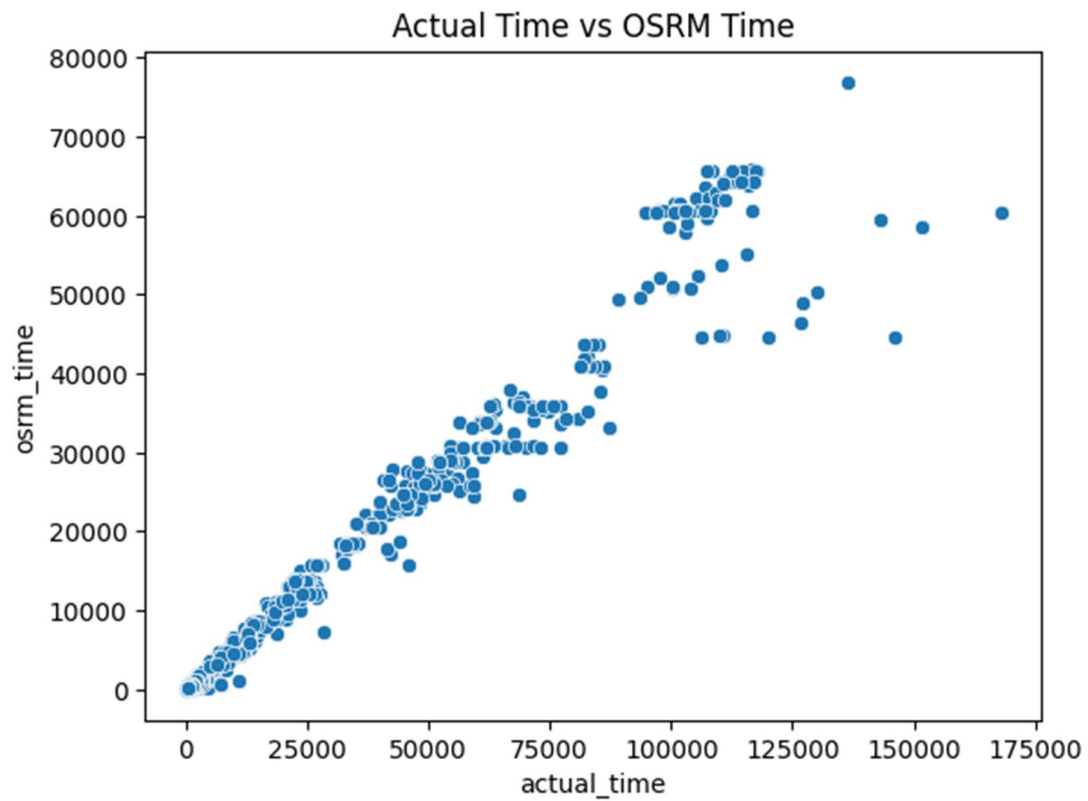
### Split Features

### Visual Comparison

### Actual Time vs OSRM Time

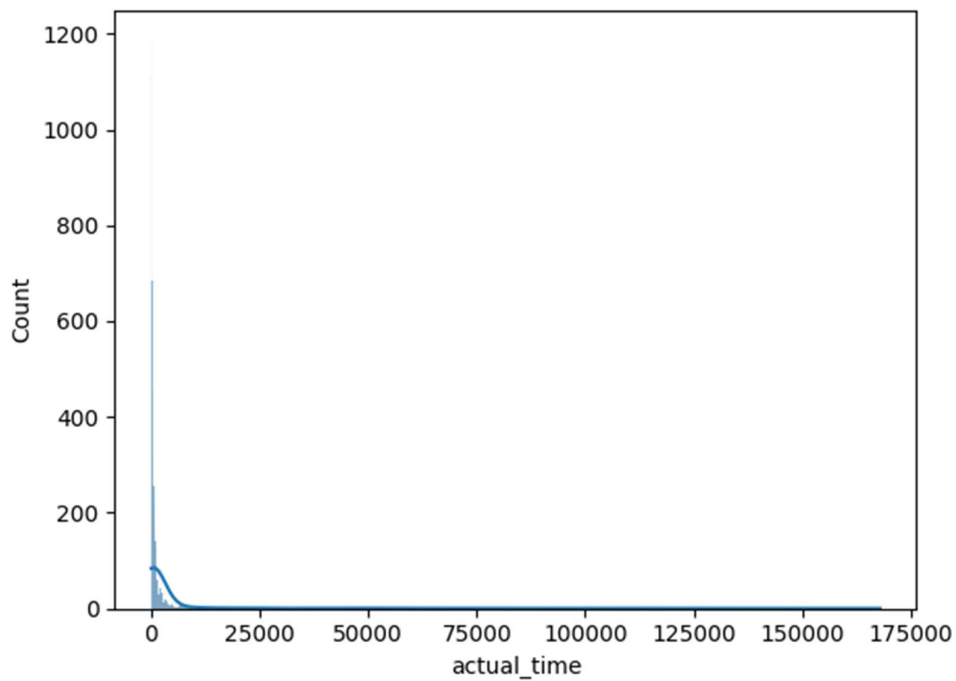
```
sns.scatterplot(
    x=trip['actual_time'],
    y=trip['osrm_time']
)

plt.title("Actual Time vs OSRM Time")
plt.show()
```



## Distribution Plot

```
sns.histplot(trip['actual_time'], kde=True)  
plt.show()
```



# Hypothesis Testing

## Actual Time vs OSRM Time

### Hypothesis

- H0: Means are equal
- H1: Means are different

```
ttest_rel(  
    trip['actual_time'],  
    trip['osrm_time']  
)
```

```
TtestResult(statistic=np.float64(21.488007537951507),  
pvalue=np.float64(5.934714921803515e-99), df=np.int64(6396))
```

## Actual Time vs Segment Actual Time

```
ttest_rel(  
    trip['actual_time'],  
    trip['segment_actual_time']  
)
```

```
TtestResult(statistic=np.float64(20.412057613657144),  
pvalue=np.float64(9.0094878546703e-90), df=np.int64(6396))
```

## OSRM Distance vs Segment OSRM Distance

```
ttest_rel(  
    trip['osrm_distance'],  
    trip['segment_osrm_distance']  
)
```

```
TtestResult(statistic=np.float64(19.982943892736152),  
pvalue=np.float64(3.1744647603731452e-86), df=np.int64(6396))
```

## OSRM Time vs Segment OSRM Time

```
ttest_rel(  
    trip['osrm_time'],  
    trip['segment_osrm_time']  
)
```

```
ttestResult(statistic=np.float64(20.161621326145493),  
pvalue=np.float64(1.0783842676580913e-87), df=np.int64(6396))
```

## Visual Analysis

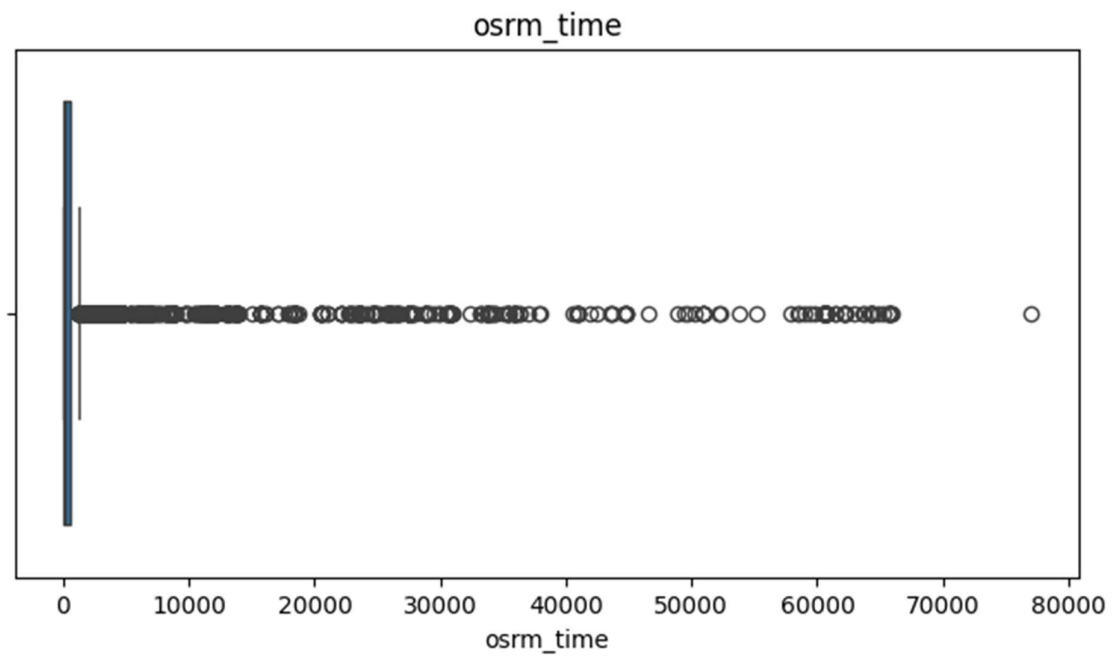
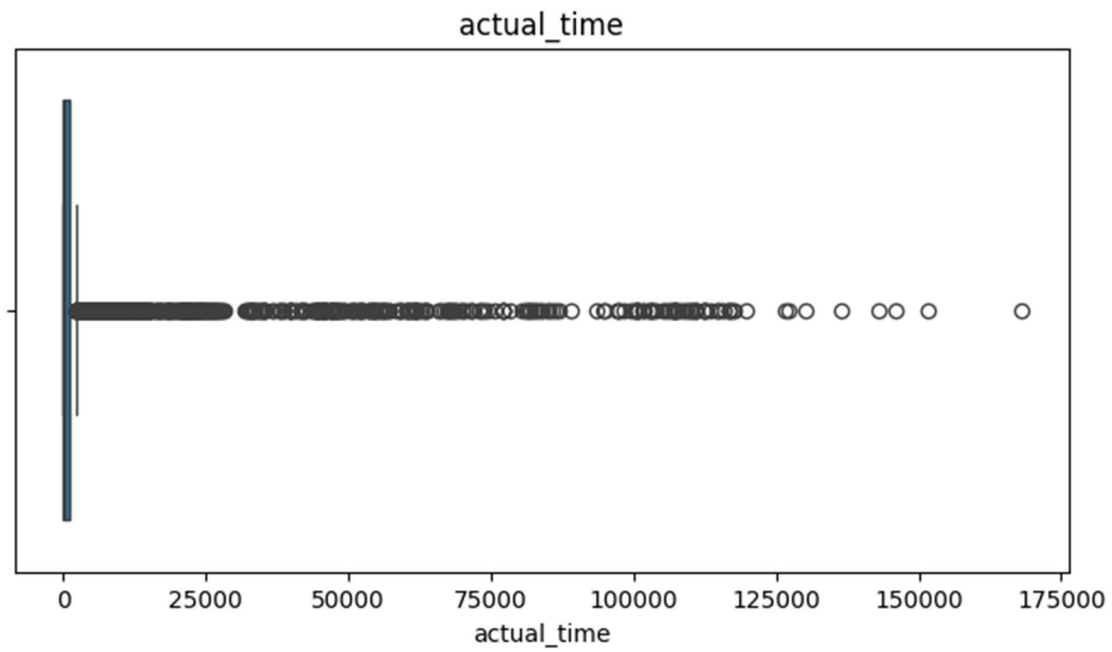
### Boxplots for Outliers

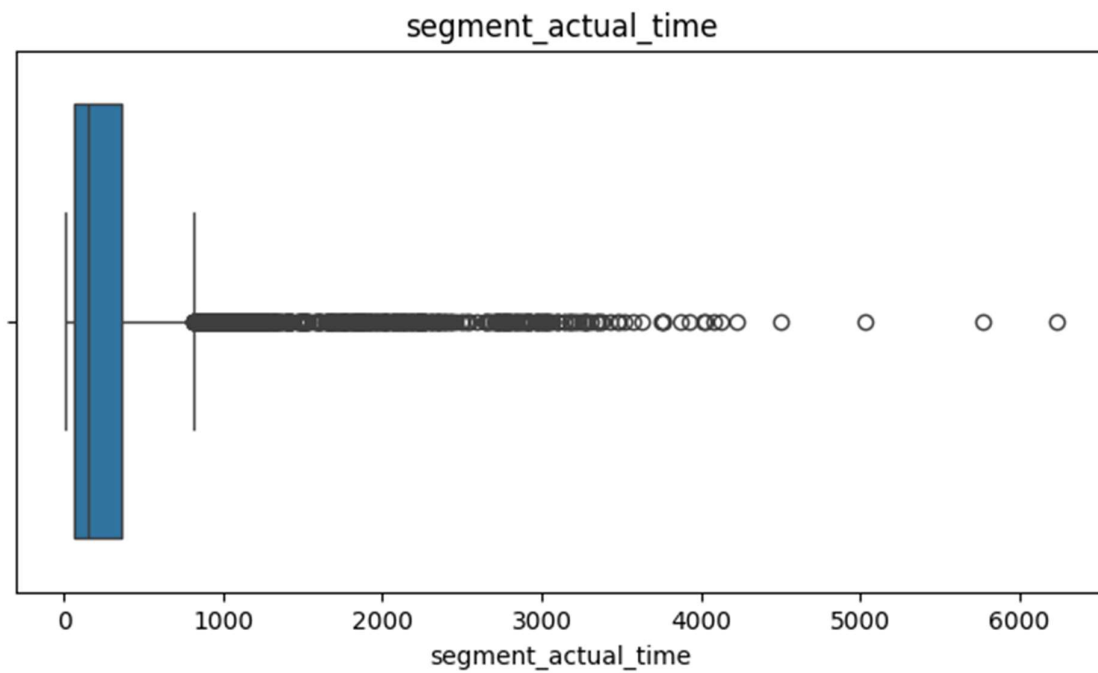
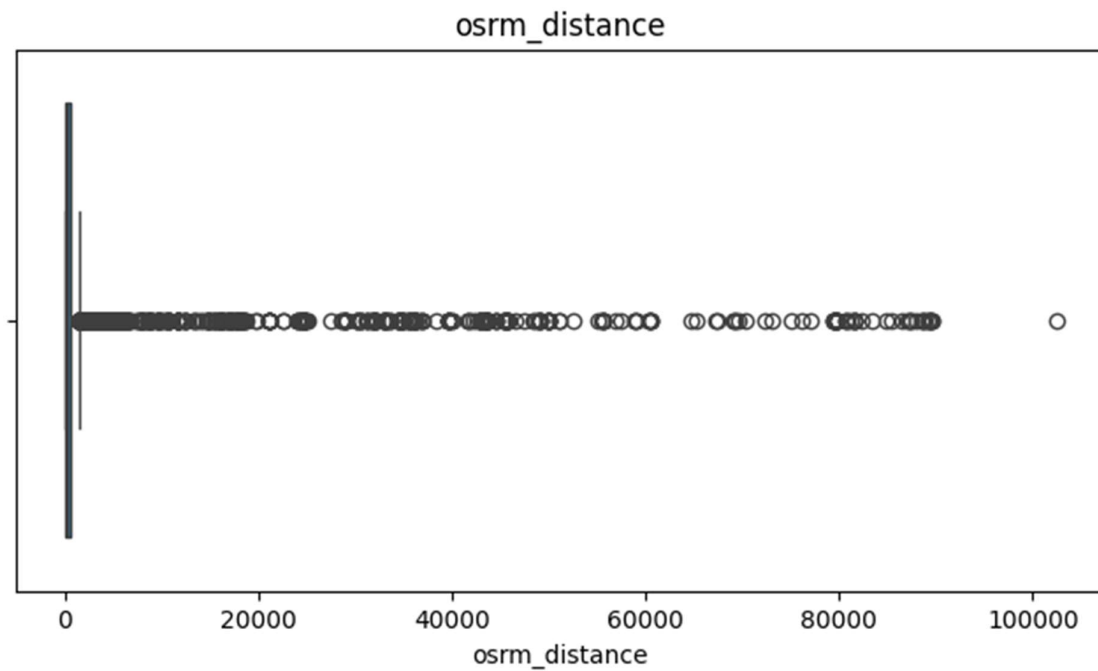
```
num_cols = [  
    'actual_time',  
    'osrm_time',  
    'osrm_distance',  
    'segment_actual_time'  
)
```

```

for col in num_cols:
    plt.figure(figsize=(8,4))
    sns.boxplot(x=trip[col])
    plt.title(col)
    plt.show()

```





## Outlier Treatment Using IQR Function

```
def handle_outliers(df, col):

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR

    df[col] = np.where(
        df[col] > upper,
        upper,
```

```

        np.where(
            df[col] < lower,
            lower,
            df[col]
        )
    )

    return df

```

```

for col in num_cols:
    trip = handle_outliers(trip, col)

```

## Handle Categorical Variables

### One-Hot Encoding

```

trip = pd.get_dummies(
    trip,
    columns=['route_type'],
    drop_first=True
)

```

## Standardization / Normalization

### StandardScaler

```

scaler = StandardScaler()

scale_cols = [
    'actual_time',
    'osrm_time',
    'osrm_distance',
    'segment_actual_time',
    'segment_osrm_time'
]

trip[scale_cols] = scaler.fit_transform(
    trip[scale_cols]
)

```

## Correlation Analysis

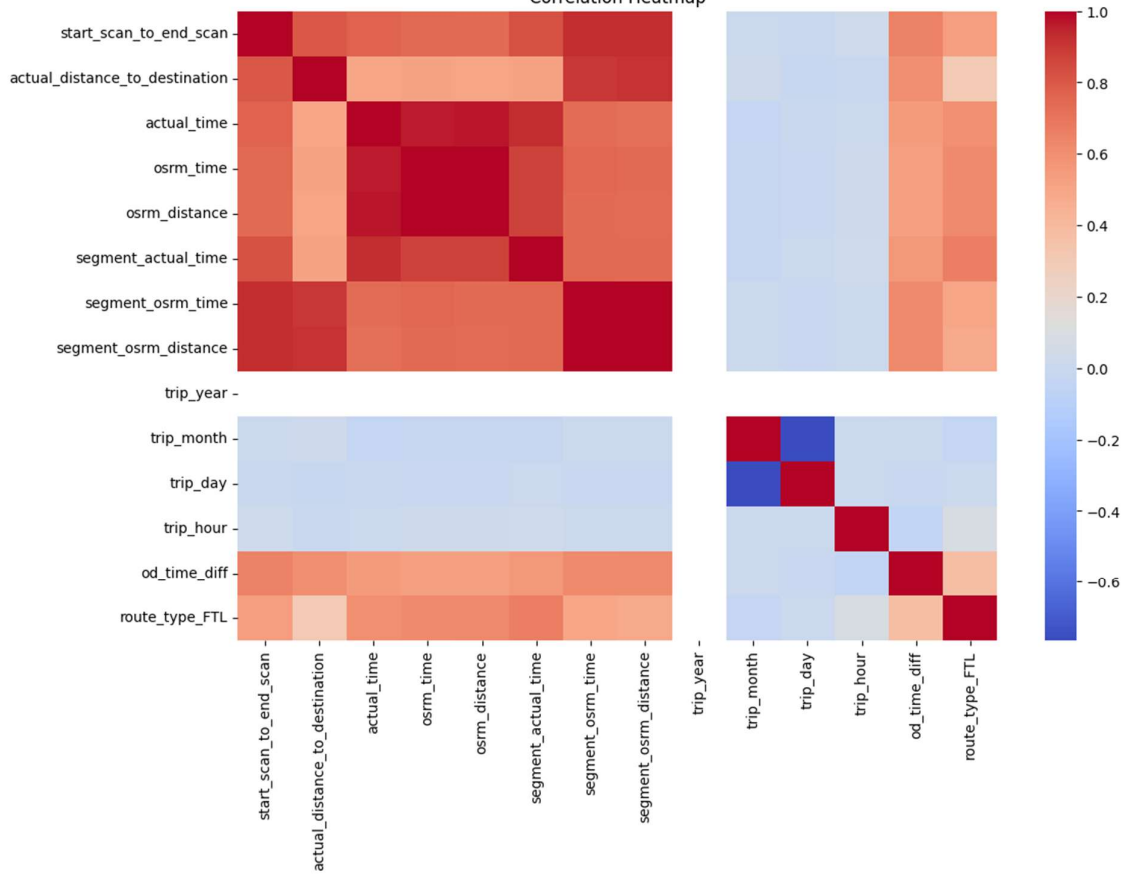
```

plt.figure(figsize=(12,8))

sns.heatmap(
    trip.corr(numeric_only=True),
    cmap='coolwarm',
    annot=False
)

plt.title("Correlation Heatmap")
plt.show()

```



## **Important EDA Insights :**

### **Time-Based Insights**

- Actual delivery time is generally higher than OSRM estimated time.
- Longer routes show higher delivery variability.
- Peak-hour trips take significantly more time.

### **Distance Insights**

- Distance and delivery time are strongly positively correlated.
- Some trips have unusually high delivery time despite shorter distance → operational inefficiencies.

### **Route Insights**

- FTL routes are generally faster and more efficient than Carting routes.
- Carting has higher delay variance due to multiple stops.

### **Operational Insights**

- Certain corridors contribute most deliveries.
- Some warehouses repeatedly show delays.

## **Business Insights**

### **Example 1**

Majority of orders originate from metro cities and industrial hubs.

### **Example 2**

Long-distance corridors show greater mismatch between OSRM predicted time and actual delivery time.

### **Example 3**

Carting routes have higher turnaround time than FTL routes.



## **Recommendations**

### **Recommendation 1**

Improve route optimization for corridors where actual time greatly exceeds OSRM estimates.

### **Recommendation 2**

Increase FTL usage for high-volume routes to reduce delays.

### **Recommendation 3**

Monitor warehouses with consistently high segment delays.

### **Recommendation 4**

Use dynamic traffic-aware routing instead of static OSRM estimates.